Functions - inputs, results, and behavior

Today

Define and call functions with arguments, then use their returned values.
Explain side effects and repeated application, and rewrite a function to be pure.

Trace and modify higher-order functions and closures, including independent state.

Follow the lesson, then pair up

First follow the slides in this notebook. Predict, run, and make a small change. After instruction, pair with an instructor and continue into practice.

function

A named set of steps you can run whenever you need them.

`def` gives these steps a name. The indented lines are the steps.

Code

```
1 def greet():
2     print("Hello")
3 greet()
```

Printed output

function

A named set of steps you can run whenever you need them.

`def` gives these steps a name. The indented lines are the steps.

Code

```
1 def greet():
2     print("Hello")
3 greet()
```

Printed output

```
Hello
```

function call

Running a function by writing its name followed by parentheses.

The marked `greet()` runs the greeting steps.

Code

```
1 def greet():
2     print("Hello")
3 greet()
```

Printed output

TERMINOLOGY

function call

Running a function by writing its name followed by parentheses.

The marked `greet()` runs the greeting steps.

Code

```
1 def greet():
2     print("Hello")
3 greet()
```

Printed output

```
Hello
```

Define a sound, then call it

Predict the output. The indented body runs when the function is called.

Code

```
1 def car_noise():
2     print("Vroom!")
3
4 car_noise()
5 car_noise()
```

Printed output

Define a sound, then call it

Predict the output. The indented body runs when the function is called.

Code

```
1 def car_noise():
2     print("Vroom!")
3
4 car_noise()
5 car_noise()
```

Printed output

```
Vroom!
Vroom!
```

EXERCISE

Exercise 0 — Define a sound, then call it

Add a `car_brakes` function that prints `"Screech!"`. Call the two functions in an order you choose.

```
def car_brakes():
    # YOUR CODE HERE
    pass
```

parameter

A name a function uses for an input you give it.

Here, `name` holds the name of the person to greet.

Code

```
1 def greet(name):
2     print("Hello", name)
3 greet("Jenna")
```

Printed output

parameter

A name a function uses for an input you give it.

Here, `name` holds the name of the person to greet.

Code

```
1 def greet(name):
2     print("Hello", name)
3 greet("Jenna")
```

Printed output

```
Hello Jenna
```

argument

The value you give a function when you call it.

Here, `"Jenna"` is the value given to the `name` parameter.

Code

```
1 def greet(name):
2     print("Hello", name)
3 greet("Jenna")
```

Printed output

argument

The value you give a function when you call it.

Here, `"Jenna"` is the value given to the `name` parameter.

Code

```
1 def greet(name):
2     print("Hello", name)
3 greet("Jenna")
```

Printed output

```
Hello Jenna
```

Give a function an input

`name` is a parameter. `"Jenna"` and `"Ryan"` are arguments supplied by two calls.

Code

```
1 def greet(name):
2     print("Hello", name)
3
4 greet("Jenna")
5 greet("Ryan")
```

Printed output

Give a function an input

name is a parameter. `"Jenna"` and `"Ryan"` are arguments supplied by two calls.

Code

```
1 def greet(name):
2     print("Hello", name)
3
4 greet("Jenna")
5 greet("Ryan")
```

Printed output

```
Hello Jenna
Hello Ryan
```

EXERCISE

Exercise 1 — Give a function an input

Complete `cheer(name, message)` and call it for two people.

```
def cheer(name, message):
    # YOUR CODE HERE
    pass
```

return value

The answer a function gives back to the code that called it.

`return` sends the answer back. `result` stores it for later use.

Code

```
1 def double(number):
2     return number * 2
3 result = double(4)
4 print(result)
```

Printed output

return value

The answer a function gives back to the code that called it.

`return` sends the answer back. `result` stores it for later use.

Code

```
1 def double(number):
2     return number * 2
3 result = double(4)
4 print(result)
```

Printed output

```
8
```

Return a result to the caller

Predict what is displayed. Returning a value lets the caller decide how to use it.

Code

```
1 def add(first, second):
2     return first + second
3
4 answer = add(10, 5)
5 print(answer)
6 print(answer * 2)
```

Printed output

Return a result to the caller

Predict what is displayed. Returning a value lets the caller decide how to use it.

Code

```
1 def add(first, second):
2     return first + second
3
4 answer = add(10, 5)
5 print(answer)
6 print(answer * 2)
```

Printed output

```
15
30
```

EXERCISE

Exercise 2 — Return a result to the caller

Write `double(number)`. Save the result of `double(6)` and print that result plus `1`.

```
def double(number):
    # YOUR CODE HERE
    pass
```

scope

The parts of your code where you can use a particular variable.

`message` belongs inside `greet`. Use **`return`** to send its value back.

Code

```
1 def greet(name):
2     message = "Hello " + name
3     return message
4 print(greet("Jenna"))
```

Printed output

scope

The parts of your code where you can use a particular variable.

`message` belongs inside `greet`. Use **`return`** to send its value back.

Code

```
1 def greet(name):
2     message = "Hello " + name
3     return message
4 print(greet("Jenna"))
```

Printed output

```
Hello Jenna
```

Names inside a function

Predict both names. Assigning the local `name` does not replace the outside `name`.

Code

```
1 name = "Jenna"
2 def make_greeting(name):
3     message = "Hello " + name
4     return message
5
6 print(make_greeting("Ryan"))
7 print(name)
```

Printed output

Names inside a function

Predict both names. Assigning the local `name` does not replace the outside `name`.

Code

```
1 name = "Jenna"
2 def make_greeting(name):
3     message = "Hello " + name
4     return message
5
6 print(make_greeting("Ryan"))
7 print(name)
```

Printed output

```
Hello Ryan
Jenna
```

side effect

Something a function does besides giving back an answer.

Printing a message is a side effect you can see on the screen.

Code

```
1 def report(number):
2     print("Checking", number)
3     return number * 2
4 result = report(4)
```

Printed output

side effect

Something a function does besides giving back an answer.

Printing a message is a side effect you can see on the screen.

Code

```
1 def report(number):
2     print("Checking", number)
3     return number * 2
4 result = report(4)
```

Printed output

```
Checking 4
```

pure function

A function that gives the same answer for the same inputs and does nothing else outside itself.
`double` only returns an answer. The separate `print()` displays it.

Code

```
1 def double(number):
2     return number * 2
3 print(double(4))
```

Printed output

pure function

A function that gives the same answer for the same inputs and does nothing else outside itself.
`double` only returns an answer. The separate `print()` displays it.

Code

```
1 def double(number):
2     return number * 2
3 print(double(4))
```

Printed output

```
8
```

determinism

Getting the same answer each time you give the same inputs.

Both calls start with `4`, so both give the same answer.

Code

```
1 def double(number):
2     return number * 2
3 print(double(4))
4 print(double(4))
```

Printed output

determinism

Getting the same answer each time you give the same inputs.

Both calls start with `4`, so both give the same answer.

Code

```
1 def double(number):
2     return number * 2
3 print(double(4))
4 print(double(4))
```

Printed output

```
8
8
```

Return text instead of printing it

This function only builds and returns text. Its caller handles display.

Code

```
1 def make_message(name):
2     return "Hello " + name
3
4 print(make_message("Jenna"))
5 print(make_message("Jenna"))
```

Printed output

Return text instead of printing it

This function only builds and returns text. Its caller handles display.

Code

```
1 def make_message(name):
2     return "Hello " + name
3
4 print(make_message("Jenna"))
5 print(make_message("Jenna"))
```

Printed output

```
Hello Jenna
Hello Jenna
```

Exercise 3 — Return text instead of printing it

Rewrite `announce` so it returns the greeting. Move printing to the caller.

```
def announce(name):
    print("Welcome " + name)
# YOUR CODE HERE
```

list

A group of values kept in order inside square brackets.

Each value is an element. `[0]` selects the first one.

Code

```
1 words = ["no", "ice"]
2 print(words)
3 print(words[0])
```

Printed output

list

A group of values kept in order inside square brackets.

Each value is an element. `[0]` selects the first one.

Code

```
1 words = ["no", "ice"]
2 print(words)
3 print(words[0])
```

Printed output

```
['no', 'ice']
no
```

A small amount of list vocabulary

A list holds values in order. `+` makes a new list here. `append()` changes the supplied list.

Code

```
1 items = ["no"]
2 combined = items + ["ice"]
3 print(items)
4 print(combined)
5 items.append("ice")
6 print(items)
```

Printed output

A small amount of list vocabulary

A list holds values in order. `+` makes a new list here. `append()` changes the supplied list.

Code

```
1 items = ["no"]
2 combined = items + ["ice"]
3 print(items)
4 print(combined)
5 items.append("ice")
6 print(items)
```

Printed output

```
['no']
['no', 'ice']

['no', 'ice']
```

impure function

A function that does more than return an answer, or uses outside information that can change.
`append()` changes the original order. Previewing twice adds two bags.

Code

```
1 def preview_order(items):
2     items.append("bag")
3     return items
4 order = ["book"]
5 print(preview_order(order))
6 print(preview_order(order))
```

Printed output

impure function

A function that does more than return an answer, or uses outside information that can change.
`append()` changes the original order. Previewing twice adds two bags.

Code

```
1 def preview_order(items):
2     items.append("bag")
3     return items
4 order = ["book"]
5 print(preview_order(order))
6 print(preview_order(order))
```

Printed output

```
['book', 'bag']
['book', 'bag',
'bag']
```

A preview that changes the order

A preview should show an order without adding items. This function changes the original list each time you look.

First preview: one bag is added.

Second preview: another bag is added.

A later checkout now sees both bags in the order.

Should viewing an order change what you buy?

idempotence

Doing something again after it is done makes no further change.

Clean the text, then clean that already-cleaned text. It stays the same.

Code

```
1 def clean(text):
2     return text.strip().lower()
3 cleaned = clean(" Hi ")
4 print(cleaned)
5 print(clean(cleaned))
```

Printed output

idempotence

Doing something again after it is done makes no further change.

Clean the text, then clean that already-cleaned text. It stays the same.

Code

```
1 def clean(text):
2     return text.strip().lower()
3 cleaned = clean(" Hi ")
4 print(cleaned)
5 print(clean(cleaned))
```

Printed output

```
hi
hi
```

Apply the operation again

Compare one application with applying the result again. Which transformation stops changing?

Code

```
1 def normalize(text):
2     return text.strip().lower()
3 def double(number):
4     return number * 2
5
6 print(normalize("  HELLO  "))
7 print(normalize(normalize("  HELLO  ")))
8 print(double(3))
9 print(double(double(3)))
```

Printed output

Apply the operation again

Compare one application with applying the result again. Which transformation stops changing?

Code

```
1 def normalize(text):
2     return text.strip().lower()
3 def double(number):
4     return number * 2
5
6 print(normalize("  HELLO  "))
7 print(normalize(normalize("  HELLO  ")))
8 print(double(3))
9 print(double(double(3)))
```

Printed output

```
hello
hello
6
12
```

EXERCISE

Exercise 4 — Apply the operation again

Choose a new text and a nonzero number. Compare one application with two applications for each operation.

```
# YOUR CODE HERE
```

higher-order function

A function that takes another function as input or gives a function back.

`apply` takes `double` as an input and runs it on the number.

Code

```
1 def double(number):
2     return number * 2
3 def apply(operation, value):
4     return operation(value)
5 print(apply(double, 4))
```

Printed output

higher-order function

A function that takes another function as input or gives a function back.

`apply` takes `double` as an input and runs it on the number.

Code

```
1 def double(number):
2     return number * 2
3 def apply(operation, value):
4     return operation(value)
5 print(apply(double, 4))
```

Printed output

```
8
```

Pass a function as an argument

`double` supplies a function. `double(3)` would supply its result instead.

Code

```
1 def double(number):
2     return number * 2
3 def apply_twice(operation, value):
4     return operation(operation(value))
5
6 print(apply_twice(double, 3))
```

Printed output

Pass a function as an argument

`double` supplies a function. `double(3)` would supply its result instead.

Code

```
1 def double(number):
2     return number * 2
3 def apply_twice(operation, value):
4     return operation(operation(value))
5
6 print(apply_twice(double, 3))
```

Printed output

```
12
```

EXERCISE

Exercise 5 — Pass a function as an argument

Define `add_one(number)` and pass it to `apply_twice` with starting value `3`. Predict the result first.

```
def apply_twice(operation, value):
    return operation(operation(value))
def add_one(number):
    # YOUR CODE HERE
    pass
```

closure

An inner function that remembers values from the function around it.

`multiply` remembers `factor`, even after `make_multiplier` finishes.

Code

```
1 def make_multiplier(factor):
2     def multiply(number):
3         return number * factor
4     return multiply
5 times_three = make_multiplier(3)
6 print(times_three(4))
```

Printed output

closure

An inner function that remembers values from the function around it.

`multiply` remembers `factor`, even after `make_multiplier` finishes.

Code

```
1 def make_multiplier(factor):
2     def multiply(number):
3         return number * factor
4     return multiply
5 times_three = make_multiplier(3)
6 print(times_three(4))
```

Printed output

```
12
```

Return a function that remembers

Each call to `make_multiplier` creates a function with its own enclosing `factor`.

Code

```
1 def make_multiplier(factor):
2     def multiply(number):
3         return number * factor
4     return multiply
5
6 double = make_multiplier(2)
7 triple = make_multiplier(3)
8 print(double(5))
9 print(triple(5))
```

Printed output

Return a function that remembers

Each call to `make_multiplier` creates a function with its own enclosing `factor`.

Code

```
1 def make_multiplier(factor):
2     def multiply(number):
3         return number * factor
4     return multiply
5
6 double = make_multiplier(2)
7 triple = make_multiplier(3)
8 print(double(5))
9 print(triple(5))
```

Printed output

```
10
15
```

EXERCISE

Exercise 6 — Return a function that remembers

Create a multiplier for `10`, then use it with `4` and `7`.

```
def make_multiplier(factor):
    def multiply(number):
        return number * factor
    return multiply
# YOUR CODE HERE
```

stateful function

A function that remembers information and updates it between calls.

This counter remembers its count and adds one each time.

Code

```
1 def make_counter():
2     count = [0]
3     def next_count():
4         count[0] = count[0] + 1
5         return count[0]
6     return next_count
7 next_count = make_counter()
8 print(next_count(), next_count())
```

Printed output

stateful function

A function that remembers information and updates it between calls.

This counter remembers its count and adds one each time.

Code

```
1 def make_counter():
2     count = [0]
3     def next_count():
4         count[0] = count[0] + 1
5         return count[0]
6     return next_count
7 next_count = make_counter()
8 print(next_count(), next_count())
```

Printed output

```
1 2
```

Independent counters remember state

Predict the three results. Each factory call creates a separate one-element list.

Code

```
def make_counter():
    count = [0]
    def next_count():
        count[0] = count[0] + 1
        return count[0]
    return next_count

first = make_counter()
second = make_counter()
print(first(), first(), second())
```

Printed output

Independent counters remember state

Predict the three results. Each factory call creates a separate one-element list.

Code

```
def make_counter():
    count = [0]
    def next_count():
        count[0] = count[0] + 1
        return count[0]
    return next_count

first = make_counter()
second = make_counter()
print(first(), first(), second())
```

Printed output

```
1 2 1
```

Exercise 7 — Independent counters remember state

Make two fresh counters. Call them in an order you choose and predict each result.

Rerun the whole cell to reset them.

```
def make_counter():
    count = [0]
    def next_count():
        count[0] = count[0] + 1
        return count[0]
    return next_count
# YOUR CODE HERE
```

Optional reference: a default value

A default value is used when an argument is omitted. Explicit arguments are our normal practice today.

Code

```
1 def greet(name="Students"):
2     return "Hello " + name
3
4 print(greet())
5 print(greet("Jenna"))
```

Printed output

Optional reference: a default value

A default value is used when an argument is omitted. Explicit arguments are our normal practice today.

Code

```
1 def greet(name="Students"):
2     return "Hello " + name
3
4 print(greet())
5 print(greet("Jenna"))
```

Printed output

```
Hello Students
Hello Jenna
```

CODE EXAMPLE

Optional reference: lambda notation

A lambda writes a small function as one expression. Prefer a descriptive named function in today's work.

```
def double(number):
    return number * 2

# Same calculation in a short expression:
double_short = lambda number: number * 2
```

Exercise 8 — Pairing starts here: repair addition

Pair with an instructor. Finish any follow-along tasks first. Repair the function so `add(4, 7)` returns `11`. Add two more calls.

```
def add(first, second):
    return first
# YOUR CODE HERE
```

Exercise 9 — Rewrite a ticket calculation to be pure

Rewrite `ticket_total` to return the total instead of printing. Store totals for two orders, then print their combined cost outside the function.

```
def ticket_total(price, quantity):
    print(price * quantity)
# YOUR CODE HERE
```

Exercise 10 — Optional: make an early decision

Write `ticket_price(age)` that returns `5` below age 18 and `8` otherwise. Check ages 17, 18, and 30. Explain why a return ends the call.

```
def ticket_price(age):
    # YOUR CODE HERE
    pass
```

Exercise 11 — Optional: rescue a local result

Complete `square(number)` and use its returned value outside the function. Explain why the caller should not try to read its local `squared_value` directly.

```
1 def square(number):
2     squared_value = number * number
3     # YOUR CODE HERE
```

EXERCISE

Exercise 12 — Optional: separate conversion and calculation

Write pure `add_three(number)`. Convert the supplied numeric text outside it, call the function, then print the result. Assume valid integer text.

```
response = "12"
def add_three(number):
    # YOUR CODE HERE
    pass
```

Exercise 13 — Optional: repair division

Complete `divide(first, second)`. Return **`None`** for a zero divisor, otherwise return the quotient. Check `10, 2` and `10, 0`.

```
def divide(first, second):
    # YOUR CODE HERE
    pass
```

EXERCISE

Exercise 14 — Optional: ensure one exclamation ending

Write `ensure_excited(text)`: leave text unchanged if it already ends in `!`, otherwise add `!`. Check an empty string too. Is this idempotent?

```
def ensure_excited(text):
    # YOUR CODE HERE
    pass
```

EXERCISE

Exercise 15 — Optional: clamp a percentage

Write `clamp(number)` that returns `0` below zero, `100` above 100, and the number otherwise. Compare one and two applications.

```
def clamp(number):
    # YOUR CODE HERE
    pass
```

Exercise 16 — Optional: test a tempting claim

A student says doubling is idempotent because doubling zero twice still gives zero. Find a counterexample. Compare repeated same-input calls with repeated application.

```
def double(number):
    return number * 2
number = 0
# YOUR CODE HERE
```

Exercise 17 — Optional: deterministic but impure

Call `report(4)` twice. It returns the same number each time. Explain its side effect, then rewrite it to be pure and put display in the caller.

```
def report(number):
    print("Checking", number)
    return number * 2
# YOUR CODE HERE
```

Exercise 18 — Optional: choose a calculation

Complete `calculate(operation, first, second)`. Pass named add and multiply functions. Use the same inputs with each operation.

```
def add(first, second):
    return first + second
def multiply(first, second):
    return first * second
def calculate(operation, first, second):
    # YOUR CODE HERE
    pass
```

Exercise 19 — Optional: transformations in order

Complete `apply_in_order(first, second, value)`. Compare adding one then doubling with doubling then adding one. Start at `3`.

```
def add_one(number):
    return number + 1
def double(number):
    return number * 2
def apply_in_order(first, second, value):
    # YOUR CODE HERE
    pass
```

EXERCISE

Exercise 20 — Optional: the function or its result

Assign `double` to the name `operation`, then call `operation(5)`. Explain why assigning `double(5)` instead would give a number rather than a callable.

```
def double(number):
    return number * 2
# YOUR CODE HERE
```

EXERCISE

Exercise 21 — Optional: make a greeter

Complete `make_greeter(greeting)` so it returns a function accepting a name. Make separate Hello and Welcome greeters.

```
def make_greeter(greeting):
    def greet(name):
        # YOUR CODE HERE
        pass
    return greet
```

Exercise 22 — Optional: make a range checker

Return a function that checks whether a number is between the supplied lower and upper bounds, inclusive. Make checkers for 0–100 and 10–20.

```
def make_range_checker(low, high):
    def in_range(number):
        # YOUR CODE HERE
        pass
    return in_range
```

EXERCISE

Exercise 23 — Optional: a counter with a starting value

Modify the counter factory to remember a supplied starting value. Create counters starting at 0 and 10. Each call increases its own count by 1.

```
def make_counter(start):
    count = [start]
    def next_count():
        # YOUR CODE HERE
        pass
    return next_count
```

EXERCISE

Exercise 24 — Optional: rewrite the counter to be pure

Write `next_count(current)` that returns the next number without remembered state. Update the caller's count explicitly. Compare this design with a closure counter.

```
def next_count(current):
    # YOUR CODE HERE
    pass
count = 0
```

EXERCISE

Exercise 25 — Optional: create a fresh list

Write `make_words()` to return a new list containing `"no"` and `"ice"`. Create two results. Append `"!"` to the first and check the second stays unchanged.

```
def make_words():
    # YOUR CODE HERE
    pass
```

Exercise 26 — Optional: preserve the supplied list

Rewrite `add_word` to return a new list with the extra word, leaving the input unchanged. Check the input after calling it twice.

```
def add_word(words, word):
    words.append(word)
    return words
# YOUR CODE HERE
```

EXERCISE

Exercise 27 — Optional: classify and defend

Classify these functions: pure or impure, and idempotent or not. Call each twice with the same input and then with its own result. Explain your reasoning.

```
def clean(text):
    return text.strip().lower()
def double(number):
    return number * 2
def announce(text):
    print(text)
    return text
# YOUR CODE HERE
```

EXERCISE

Exercise 28 — Optional: refactor an order calculation

Rewrite `order_total` to return the discounted total and move printing outside. Then make it accept a discount function instead of calculating the discount itself.

```
def order_total(price, quantity):
    subtotal = price * quantity
    print(subtotal * 0.9)
# YOUR CODE HERE
```

Exercise 29 — Optional: a discount factory

Write `make_discount(rate)` that returns a function reducing an amount by that rate. Create 10% and 20% discounts. Assume a rate between 0 and 1.

```
def make_discount(rate):
    def discount(amount):
        # YOUR CODE HERE
        pass
    return discount
```